



INSTITUTO POLITÉCNICO NACIONAL.

ESCUELA SUPERIOR DE CÓMPUTO.

REPORTE DE PRÁCTICA:

Instalación y Configuración de Mosquitto MQTT

en Raspberry Pi con Publisher ESP32

ALUMNO:

MORAN VAQUERO MARCOS



1. Introducción

El protocolo MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería ligero basado en el modelo publicador/suscriptor (pub/sub), ampliamente utilizado en aplicaciones de Internet de las Cosas (IoT) por su bajo consumo de ancho de banda y su eficiencia en redes con recursos limitados.

En esta práctica se llevó a cabo la instalación y configuración del broker Mosquitto en una Raspberry Pi, la cual actuó como servidor central de mensajería. Posteriormente, se programó una tarjeta ESP32 como publicador (Publisher), enviando mensajes periódicos al broker a través de la red local. Finalmente, se verificó la recepción de dichos mensajes mediante un cliente suscriptor (Subscriber).

2. Objetivos

- Comprender el protocolo MQTT y su aplicación en arquitecturas IoT.
- Instalar y configurar el broker Mosquitto en Raspberry Pi.
- Programar un Publisher en ESP32 para enviar mensajes periódicos al broker.
- Verificar la recepción de mensajes mediante un cliente Subscriber.
- Documentar el proceso con buenas prácticas de desarrollo.

3. Marco Teórico

3.1 Protocolo MQTT

MQTT es un protocolo de mensajería estándar para IoT diseñado para conexiones con ancho de banda limitado y alta latencia. Opera sobre TCP/IP y utiliza un modelo de publicación/suscripción donde los dispositivos se comunican a través de un intermediario llamado broker, sin necesidad de conexión directa entre ellos.

Los elementos principales del protocolo son:

- Broker: Servidor central que recibe todos los mensajes y los distribuye a los clientes suscritos al tópico correspondiente.
- Publisher (Publicador): Cliente que envía mensajes a un tópico específico en el broker.
- Subscriber (Suscriptor): Cliente que se suscribe a uno o más tópicos y recibe los mensajes publicados en ellos.
- Tópico (Topic): Cadena jerárquica que identifica el canal de comunicación (ej. iot/mensaje).

3.2 Mosquitto MQTT Broker

Eclipse Mosquitto es una implementación de código abierto del broker MQTT, compatible con las versiones 3.1, 3.1.1 y 5.0 del protocolo. Es ligero, eficiente y ampliamente utilizado en proyectos IoT, especialmente en hardware con recursos limitados como la Raspberry Pi.



3.3 ESP32 y la librería PubSubClient

La ESP32 es un microcontrolador de doble núcleo con conectividad Wi-Fi y Bluetooth integrada, ampliamente adoptado en proyectos IoT. La librería PubSubClient para Arduino permite a la ESP32 actuar como cliente MQTT, publicando y suscribiéndose a tópicos en un broker de forma sencilla.

4. Procedimiento

4.1 Instalación del Broker Mosquitto

Se instaló Mosquitto en la Raspberry Pi ejecutando los siguientes comandos en la terminal:

```
sudo apt-get update
sudo apt-get install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Se configuró Mosquitto para permitir conexiones externas editando el archivo de configuración y añadiendo las siguientes directivas:

```
listener 1883
allow_anonymous true
```

4.2 Prueba básica del Broker

Para verificar el funcionamiento del broker, se ejecutó un Subscriber en la Raspberry Pi y se publicó un mensaje de prueba desde otra terminal:

```
# Terminal 1 - Subscriber
mosquitto_sub -h localhost -t iot/mensaje

# Terminal 2 - Publisher de prueba
mosquitto_pub -h localhost -t iot/mensaje -m "Prueba de broker"
```

Se verificó que el mensaje fue recibido correctamente por el Subscriber, confirmando el correcto funcionamiento del broker.

4.3 Configuración del Publisher en ESP32

Se programó la ESP32 utilizando el entorno Arduino IDE con la librería PubSubClient instalada. El código configura la ESP32 para conectarse a la red Wi-Fi local y publicar mensajes periódicos cada 5 segundos en el tópico iot/mensaje hacia el broker en la IP 192.168.1.76.

5. Código Fuente — ESP32 Publisher



El siguiente código implementa el Publisher MQTT en la ESP32:

```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "GENIAL64";
const char* password = "*****";
const char* mqtt_server = "192.168.1.76";
WiFiClient espClient;
PubSubClient client(espClient);
void setup_wifi() {
  Serial.println("Conectando a WiFi...");
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println(WiFi.localIP());
}
void reconnect() {
  while (!client.connected()) {
    if (client.connect("ESP32Client")) {
      Serial.println("Conectado");
    } else { delay(2000); }
  }
}
void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, 1883);
}
void loop() {
  if (!client.connected()) reconnect();
  client.loop();
  client.publish("iot/mensaje", "Hola desde ESP32");
  Serial.println("Mensaje enviado");
  delay(5000);
}
```

Nota: La contraseña Wi-Fi fue omitida por seguridad en la documentación. En el repositorio se utiliza un archivo de variables de entorno o se solicita al usuario configurarla localmente.



6.1 Instalación y ejecución del Broker Mosquitto

```

Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\nonba>mosquitto -c "C:\Users\nonba\OneDrive\Documentos\mosquitto.conf" -v
1776827986: mosquitto version 2.1.2 starting
1776827986: Config loaded from C:\Users\nonba\OneDrive\Documentos\mosquitto.conf.
1776827986: Bridge support available.
1776827986: Persistence support available.
1776827986: TLS support available.
1776827986: TLS-PSK support available.
1776827986: Websockets support available.
1776827986: Opening ipv6 listen socket on port 1883.
1776827986: Opening ipv4 listen socket on port 1883.
1776827986: mosquitto version 2.1.2 running
1776827988: New connection from 192.168.1.65:49759 on port 1883.
1776827988: New client connected from 192.168.1.65:49759 as ESP32Client (p4, c1, h15).
1776827988: No will message specified.
1776827988: Sending CONNACK to ESP32Client (0, 0)
1776827988: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776827993: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776827998: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776828003: Received PINGREQ from ESP32Client
1776828003: Sending PINGRESP to ESP32Client
1776828003: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776828008: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776828013: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776828018: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))
1776828023: Received PINGREQ from ESP32Client
1776828023: Sending PINGRESP to ESP32Client
1776828023: Received PUBLISH from ESP32Client (d0, q0, r0, n0, 'iot/mensaje', ... (16 bytes))

```

6.2 Ejecución del Publisher en ESP32 (Serial Monitor)

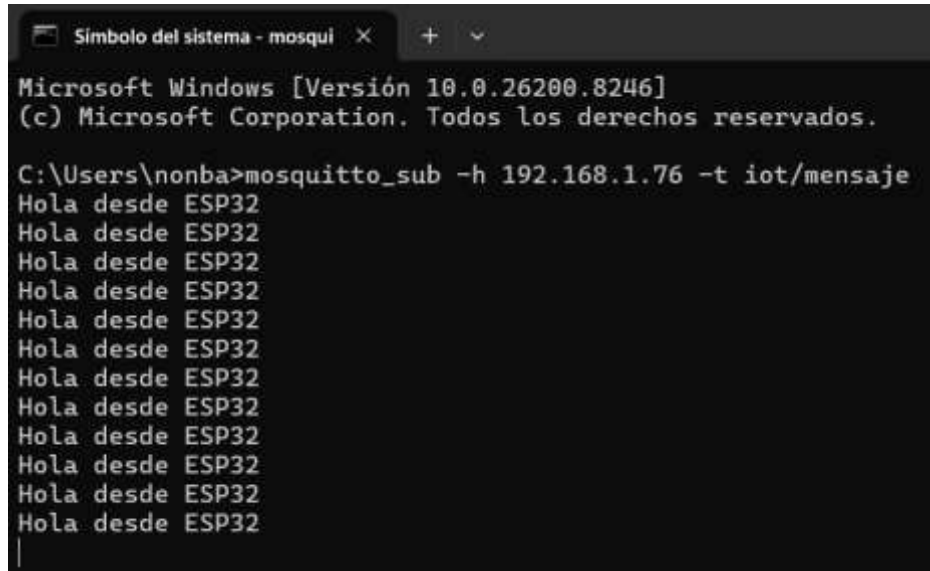
[illegible]



Figura 2. Serial Monitor de la ESP32 mostrando la conexión MQTT y el envío continuo de mensajes.

6.3 Recepción de mensajes como Subscriber

Mediante el cliente mosquitto_sub conectado al broker en la IP 192.168.1.76 y suscrito al tópico `iot/mensaje`, se reciben los mensajes publicados por la ESP32 en tiempo real:



```
Símbolo del sistema - mosqui
Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\nonba>mosquitto_sub -h 192.168.1.76 -t iot/mensaje
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
Hola desde ESP32
|
```

Figura 3. Cliente Subscriber recibiendo el mensaje "Hola desde ESP32" de forma continua.

7. Análisis de Resultados

Los resultados obtenidos demuestran el correcto funcionamiento del sistema MQTT implementado:

- El broker Mosquitto se inició correctamente en el puerto 1883, con soporte para TLS, WebSockets y persistencia habilitados.
- La ESP32 se conectó exitosamente al broker bajo el identificador `ESP32Client` y comenzó a publicar mensajes en el tópico `iot/mensaje` cada 5 segundos.
- El broker registró las publicaciones con 16 bytes de carga útil, lo que corresponde al mensaje "Hola desde ESP32".
- El cliente Subscriber recibió los mensajes de forma inmediata y continua, verificando el flujo completo Publisher → Broker → Subscriber.
- El mecanismo de keep-alive PINGREQ/PINGRESP funcionó correctamente, manteniendo la conexión activa entre la ESP32 y el broker.

8. Conclusión



Esta práctica permitió comprender de manera práctica el funcionamiento del protocolo MQTT en un entorno IoT real. Se logró configurar exitosamente un broker Mosquitto, programar una ESP32 como publicador y verificar la recepción de mensajes a través de un cliente suscriptor.

El protocolo MQTT demostró ser altamente eficiente para la comunicación entre dispositivos embebidos y servidores ligeros, con una integración sencilla gracias a la librería PubSubClient. La arquitectura pub/sub facilita la escalabilidad del sistema, permitiendo agregar múltiples publicadores y suscriptores sin modificar la lógica del broker.

Los conocimientos adquiridos sientan las bases para el desarrollo de sistemas IoT más complejos, como el monitoreo de sensores, automatización del hogar y aplicaciones industriales.

9. Documentación en GitHub

El repositorio incluye los siguientes archivos y secciones:

- `mosquitto.ino` — Código fuente completo de la ESP32.
- `README.md` — Instrucciones detalladas de instalación, configuración y uso.
- `.gitignore` — Exclusión de archivos temporales y credenciales.
- Carpeta `/evidencias` — Capturas de pantalla del funcionamiento.

Estructura del `README.md`

El archivo `README.md` incluye:

- Descripción del proyecto y objetivo.
- Requisitos de hardware y software.
- Pasos para instalar Mosquitto en Raspberry Pi.
- Instrucciones para configurar y cargar el sketch en la ESP32.
- Comandos para probar el broker con `mosquitto_pub` y `mosquitto_sub`.
- Capturas de pantalla de evidencia.

10. Referencias

Eclipse Foundation. (2024). Eclipse Mosquitto — An open source MQTT broker. <https://mosquitto.org>

Knolleary. (2024). PubSubClient library for Arduino. <https://github.com/knolleary/pubsubclient>

OASIS. (2019). MQTT Version 5.0 — OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

Espressif Systems. (2024). ESP32 Technical Reference Manual. <https://www.espressif.com/en/support/documents/technical-documents>